

1 Introduction

Escrow is the primitive that makes a tokenized cross-border payment conditional: a buyer commits funds, a seller delivers, and a third party refers the outcome to a rule. In a fiat corridor that rule is usually the correspondent bank's. On a smart-contract platform the rule is the contract, and the contract's lifecycle *is* the trust model. This paper takes that observation literally: we treat the lifecycle of an escrow contract as the object of study, and we look at the cost and control flow *between* the operations that documentation and per-operation threat models normally enumerate separately.

The subject is a Soroban (Stellar) token-escrow contract family, implemented in Rust against `soroban-sdk` 26.1.1, with a second sibling repository against 27.0.6. The family is a reasonable specimen rather than a convenient one: it is a working, tested, testnet-deployed escrow with a fee mechanism, a bounded error type, committed test snapshots, and a written dispute policy. Its documentation makes explicit, checkable economic claims. That combination is what allows the audit reported here to be unusually direct: the repository records its own on-chain fee data, so the cost of each operation is a matter of record rather than of modelling.

Three findings follow from reading that record together with the source.

First, lifecycle cost is not uniform, and the documentation says it is. Across the seven recorded steps of one observed lifecycle, `fund_escrow` cost 366,938 stroops and `release_funds` cost 17,791 stroops: a ratio of 20.6. The three escrow operations together account for 47.6% of the 770,179 stroops that this one lifecycle cost, and a single operation, `fund_escrow`, accounts for 47.6% of that lifecycle total on its own. The project's own README asserts “~\$0.00001 per transaction” and that gas is “virtually \$0”. A flat per-transaction figure cannot be approximately correct for a lifecycle whose recorded fees span a factor of 40.1. We do not restate a dollar figure of our own, because the audit materials contain no exchange rate with which to convert stroops; we make only the structural claim, which requires no conversion.

Second, the dispute mechanism is one-sided. `raise_dispute` is callable by the buyer or the seller, so entry is symmetric. Resolution is not: in state `Disputed`, `release_funds` rejects and `refund_buyer` rejects, and the only accepted transition, `auto_refund_if_expired`, returns the full amount to the buyer with no fee. The repository documents this as deliberate. It does not analyse the economic consequence, which is that the contract contains no path in which a disputed escrow pays its seller, and that a counterparty pricing this contract rationally would decline to trade.

Third, the test suite reports authorization coverage it does not have. Every test calls `env.mock_all_auths()`. Under mocked authorization, `caller.require_auth()` succeeds for any caller, so a test that expects an authorization failure cannot fail for that reason. The suite nonetheless contains a test named `test_unauthorized_refund` that passes, and no test anywhere in the family asserts that `release_funds` rejects the seller.

These three findings are one finding. The operation that carries the lifecycle's cost is not the operation the threat model interrogates; the transition that decides who is paid is a terminal transition, not the dispute entry that gets documented; and the property that would have caught both — that only the right principal may move funds — is precisely the property the standard Soroban test idiom disables. We state our contribution narrowly. We have executed one test suite per repository, read one contract, and inspected one set of on-chain fee records at $n = 1$ per step. We claim a set of specific defects and one methodological consequence, and we claim nothing about settlement speed, cost savings against any payment rail, mainnet behaviour, throughput, or production readiness, because the audit materials contain no instrumentation for any of those quantities (Section 8).

2 Background: Soroban Escrow Mechanics

2.1 Why the lifecycle, and not the transaction, is the unit of analysis

In a fee model that is uniform per transaction, the cost of a payment protocol is a function of how many transactions it needs, and the design question is “how few?”. On Soroban the cost of a transaction is not uniform across operations that a protocol author would consider interchangeable, because the operations differ in what they must read, what they must write, and which entries they must create. `fund_escrow` moves value into the contract and creates a trustline held by the contract; `release_funds` moves value out of a trustline that already exists. A designer who reasons per operation sees four operations and prices them identically. A designer who reasons per lifecycle sees that the lifecycle is dominated by its most expensive step, and that the step which dominates it is the one that commits the buyer's money.

The same argument applies to control flow. Escrow correctness is a property of the transition relation, not of the guards on individual operations. An operation can be individually correct and still be part of a lifecycle that strands funds, because a guard that is correct in isolation may be the last reachable transition.

2.2 Operations, fee arithmetic, and the observable guards

The contract family under study exposes four mutating operations. We describe them only to the depth the source supports.

`create_escrow` instantiates the record. `fund_escrow` transfers the escrowed amount from the buyer and places the contract in `Funded`. `release_funds` pays the seller and pays a protocol fee to a treasury address. `refund_buyer` returns the amount to

Table 1: Observed operation guards. The final column records what the audit materials do not establish; the state type was not fully enumerated.

Operation	Location	Accepted input state	Not established
<code>release_funds</code>	<code>lib.rs:186</code>	not Disputed	target state
<code>require_auth</code>	<code>lib.rs:231</code>	—	enclosing function
<code>caller-identity check</code>	<code>lib.rs:239</code>	—	enclosing function
<code>refund_buyer</code>	<code>lib.rs:243</code>	not Disputed	target state
<code>raise_dispute</code>	<code>lib.rs:259</code>	not Disputed	prior states
<code>auto_refund_expired</code>	<code>lib.rs:311</code>	Funded or Disputed	deadline value, clock source

Table 2: Testnet escrow proofs. Three documents, three contract addresses. The per-transaction fee data used in Section 5 was recorded against the first of these, which is not the address the README calls current.

Proof document	Asset	Amount	Contract
E2E-PROOF.md	BREAD, self-issued, 7 dp	20 BREAD(200,000,000)	CB7I2GUR...
E2E-USDC-PROOF.md	Circle USDC	5 USDC (50,000,000)	CBY6UC4I...
E2E-FEE-PROOF.md	Circle USDC	0.05 USDC (500,000)	CAFG7VWE...

the buyer. A fifth entry point, `raise_dispute`, moves an escrow into Disputed, and `auto_refund_if_expired` discharges an expired one.

The fee is computed in integer arithmetic with truncation toward zero:

$$\text{fee} = \left\lfloor \frac{\text{amount} \times \text{fee_bps}}{10\,000} \right\rfloor, \quad \text{fee_bps} \in [0, 1000]. \quad (1)$$

The upper bound `MAX.FEE.BPS` = 1000 is a 10% cap (`lib.rs:11`); the rate used in the test suite is `DEFAULT.FEE.BPS` = 80, a 0.8% rate (`tests.rs:12`). The formula appears at `lib.rs:199` and `lib.rs:226`. The remainder of the amount goes to the seller. Equation 1 has no minimum-fee term, which matters for small amounts (Finding 6.3).

Table 1 records the guard on each operation as observed in the source. The last column is load-bearing: it states what the audit materials do *not* establish, so that the transition relation is not over-read.

2.3 Testing idiom and the mocked authorization oracle

The contract family is tested with the standard Soroban host test harness. Every test begins by calling `env.mock_all_auths()`, which installs an authorization mock: `require_auth` and `require_auth_for_args` return success for any caller and any argument, while recording the invocation. This is the correct default for tests whose subject is business logic, because it lets a test set up state without constructing signatures. It is the wrong oracle for a test whose subject is authorization itself, because it removes the condition under test. A test written against the mock can only ever fail for reasons other than the one it names. Section 6.4 returns to the consequences.

3 System under Study

3.1 Contract shape

The contract under study is a single-token escrow. Its error type `EscrowError` has ten variants (`lib.rs:15--33`), among them a fee-validation variant, `InvalidFee`, raised outside the accepted $0 \leq \text{fee_bps} \leq 1000$ band. The contract is distributed as `public/escrow.wasm`, 24,827 bytes, SHA-256 `a648e75a76dae8a8b04ab08fc60d2588c68a2ffb1e043cdd106ce910a777d2c1`. The build artifact that produced that hash is not committed to the repository, so the deployed bytecode's byte-identity cannot be checked from the repository alone; the hash is recorded here as the audit's own observation.

The test suite comprises 26 committed `test_snapshots` JSON files under `contracts/contracts/escrow/test_snapshots/tests/`, generated by `soroban-sdk` 26.1.1.

3.2 Deployment evidence and version drift

Three testnet proof documents record three different contract addresses for this family, which is the clearest single indication that the deployed artifact changed across the audit period.

The README identifies the third address, `CAFG7VWENECKXXSRH64G536IG4CD3ALJ03U6HUAN4P5JPMC4EX0UPHK0`, as the current contract, at version 9. The fee figures in Table 3 were recorded against `CB7I2GURQDV4Q7YAT2PZAG3ZNFQJ37GI6MWZ4P3W7`

Table 3: Recorded testnet fees for one observed escrow lifecycle, $n = 1$ per step. Source: E2E-PROOF.md:70--95, contract CB7I2GUR. . . . Ledger numbers are sequence numbers, not timestamps.

Step	Fee (stroops)	Ledger
S1 <code>deploy_asset_and_sac</code>	183,215	4,869,181
S2 <code>buyer_trust_sac</code>	14,202	4,869,189
S3 <code>seller_trust_sac</code>	14,202	4,869,192
S4 <code>mint</code>	9,140	4,869,199
S5 <code>create_escrow</code>	164,691	4,869,212
S6 <code>fund_escrow</code>	366,938	4,869,217
S7 <code>release_funds</code>	17,791	4,869,223

SUCJZPSHP2G6L6J. This matters for Finding 6.1: the ratio reported below is a property of one observed lifecycle on an earlier address of the same family, and has not been re-measured against the address the project ships as current. Only the first proof carries a date, 2026-09-25; the other two carry none in the audit materials.

The E2E-FEE-PROOF.md document reports a conservation check on the 0.05 USDC escrow: $0.0124 + 5.0496 + 0.0004 + 0 = 5.0624$. The arithmetic is correct. The per-term semantics are not established, so the paper does not interpret which term, if any, is a protocol fee, or at what rate.

4 Method

4.1 Reproduction procedure

The audit re-ran each repository’s own suite and read the artifacts the repositories commit. No test was written to make a finding pass, and no failing test was modified. Commands and toolchain versions are given in Section 9 and in the repository README.md.

4.2 Analytic derivations

Four derivations are used in this paper, and each is a plain arithmetic identity over quantities already in the audit materials, shown inline at the point of use:

- (1) the ratio of two recorded fees, $\text{fee}(\text{fund_escrow})/\text{fee}(\text{release_funds})$;
- (2) the total of the seven recorded fees, and the share of that total contributed by one escrow operation;
- (3) the ratio of the largest to the smallest recorded fee across the lifecycle;
- (4) the largest amount for which Equation 1 truncates to zero at the default rate, which is $\lfloor (10\,000 - 1)/80 \rfloor = 124$ base units.

The figure in this paper is generated by a script that reads the recorded fee table as a committed CSV and re-checks each of these derivations against the artifact values before emitting the figure; the script aborts if any derivation disagrees (Section 9).

5 Measurements

5.1 On-chain fees, $n = 1$ per step

Table 3 reproduces the only per-transaction fee data that exists for this family: the seven recorded steps of a single escrow lifecycle on Stellar testnet, transcribed from E2E-PROOF.md:70--95. Each row is one observed transaction. There is no replication, so these are descriptions of one lifecycle, not samples from a distribution; no confidence interval, variance, or percentile is computable from them and none is reported.

Figure 1 plots the same data. Three observations follow directly, and each is arithmetic over Table 3:

$$\frac{\text{fee}(\text{fund_escrow})}{\text{fee}(\text{release_funds})} = \frac{366,938}{17,791} = 20.62 \approx 20.6\times, \quad (2)$$

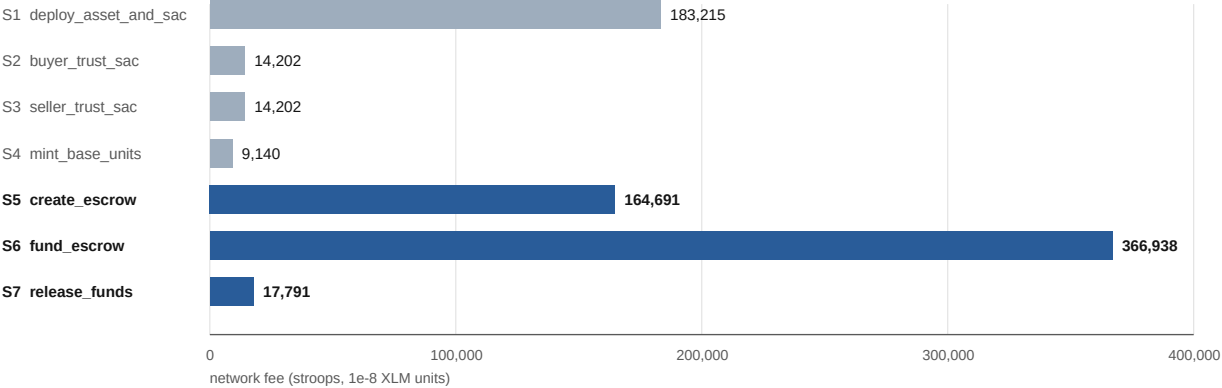
$$\sum_{i=1}^7 \text{fee}(S_i) = 770,179 \text{ stroops}, \quad (3)$$

$$\frac{\text{fee}(\text{fund_escrow})}{\sum_{i=1}^7 \text{fee}(S_i)} = 0.4764 \approx 47.6\%, \quad (4)$$

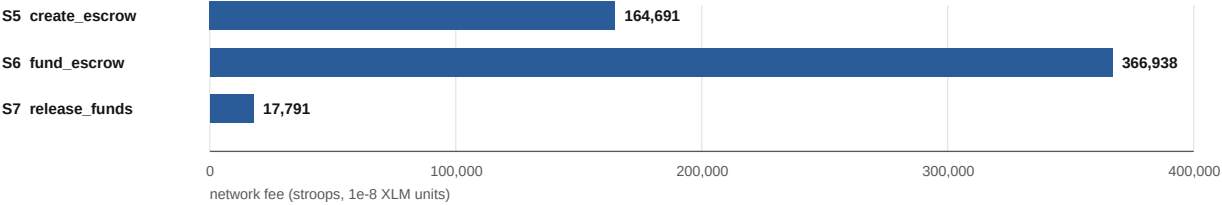
$$\frac{\max_i \text{fee}(S_i)}{\min_i \text{fee}(S_i)} = \frac{366,938}{9,140} = 40.15 \approx 40.1 \times. \quad (5)$$

Soroban escrow lifecycle fees on Stellar testnet

Contract CB7I2GURQDV4Q7YAT2PZAG3ZNFQJ37GI6MWZ4P3W7SUCJZPSHP2G6L6J
Single observed lifecycle, $n=1$ per step. Source: E2E-PROOF.md:70-95, Stellar testnet, single observed lifecycle ($n=1$).
(a) All seven recorded lifecycle steps. These data contain no wall-clock timing, so no duration is claimed here.



(b) Escrow operations only (setup steps excluded)



$\text{fund_escrow} / \text{release_funds} = 366,938 / 17,791 = 20.6\times$
Escrow operations account for 47.6% of the 770,179 stroop total of this lifecycle; fund_escrow alone is the largest single item.

escrow contract operation bootstrap / onboarding step

Figure 1: Recorded network fees for the seven steps of one observed escrow lifecycle on Stellar testnet, $n = 1$ per step. Panel (b) isolates the three escrow operations. Derived from E2E-PROOF.md:70--95; generated by figures/make_figures.py from figures/data/lifecycle_fees.csv. Because $n = 1$, no statement about fee variation is made or supported.

The absolute difference between the two escrow transfers is $366,938 - 17,791 = 349,147$ stroops. The two operations also differ from each other in direction, and the paper makes no claim about whether the asymmetry would persist at other amounts, other assets, or other addresses.

The ledger column in Table 3 is an *ordinal* sequence number. The seven steps span 42 ledger numbers, and the audit materials contain no wall-clock timing of any kind. Ledger sequence numbers cannot be converted to durations from the available data, and this paper reports no duration.

5.2 Executed tests

Table 4 lists every test command executed during the audit. A failing command is reported as a failing command.

Seventy-two tests passed in total: 26, 38, and 8. The **proofdrop** suite is 100% mocked, runs at $n = 1$, and uses an in-memory store. The **pactopay** result is the informative one: its Rust suite is green while its TypeScript integration and end-to-end suites fail, which is consistent with the two liveness defects that the Rust suite does not cover (Section 6.1 and Section 8). No green continuous integration run exists for any of the four repositories.

Table 4: Executed test commands. ✓ marks a passing run; failures are reported with their counts. Totals refer only to passing runs.

Repository	Command	Outcome	SDK	Runtime	Note
breadline-protocol	cargo test	✓ 26 passed, 0 failed	26.1.1	0.91 s	escrow contract suite
proofdrop	npm test	✓ 38 passed, 0 failed	—	—	shared 8/8, api 30/30
pactopay	cargo test	✓ 8 passed, 0 failed	27.0.6	0.63 s	contract suite only
pactopay	npx vitest run	failed 9 of 11 files, 5 of 20 tests	—	—	integration
pactopay	npx playwright test	failed 8 failed, 4 skipped, 14 passed	—	—	end-to-end
niko-sun-stellar	not executed	—	—	—	toolchain time ceiling

5.3 Test snapshots and the testnet records

Twenty-six `test_snapshots` JSON files are committed. Three testnet proof documents record escrow executions against three contract addresses (Table 2). These are the artifacts that exist; the audit records no mainnet execution of any kind.

6 Findings

6.1 F1: Lifecycle cost asymmetry invalidates the flat per-transaction cost claim

Evidence. `fund_escrow` cost 366,938 stroops and `release_funds` cost 17,791 stroops in the same lifecycle (Table 3), a ratio of 20.6 by Equation 2. The largest-to-smallest spread across the seven recorded fees is 40.1 (Equation 5). The three escrow operations account for 47.6% of the lifecycle’s 770,179 stroops (Equations 3 and 4), and `fund_escrow` alone is 47.6% of that total.

Consequence. The project’s documentation asserts “~\$0.00001 per transaction” and that gas is “virtually \$0”. We reproduce those figures here only as claims the audit falsifies, and the falsification is structural: a single per-transaction constant cannot be approximately correct for a lifecycle whose own recorded fees span a factor of 40.1. The economic content of this finding is not that fees are high — on testnet, with the amounts in Table 3, the absolute magnitudes are small — but that a cost-uniformity assumption is false, and that it is the assumption a cost model built on it would rest on. We do not restate a dollar figure of our own, because converting stroops to a fiat amount requires an exchange rate that the audit materials do not contain.

The design consequence is specific. The step that commits the buyer’s money is the expensive step: 2.2 times `create_escrow` and 20.6 times `release_funds`. A protocol that splits a payment into many small escrows pays this cost once per escrow, while a protocol that batches pays it once. A cost model that prices all four operations identically predicts none of this.

Two scope limits are load-bearing. First, the measurement is $n = 1$ per step on one address of the family, not the address the README ships as current (Table 2); the ratio is therefore version-scoped. Second, the recorded fee is an aggregate. The audit materials do not decompose it into resource classes, so this finding identifies which operation is expensive and does not claim why.

6.2 F2: One-sided dispute resolution is a dispute-extraction surface

Evidence. `raise_dispute` (`lib.rs:259`) is callable by the buyer or the seller and sets state `Disputed`. In `Disputed`, `release_funds` (`lib.rs:186`) and `refund_buyer` (`lib.rs:243`) both reject. `auto_refund_if_expired` (`lib.rs:311`) accepts `Funded` or `Disputed` and returns the amount to the buyer, and the test suite records that the returned amount carries no fee. The repository documents the behaviour as deliberate at `lib.rs:296--303`. Concatenating the three guards, there is no reachable state in which a disputed escrow pays its seller.

Consequence. Entry to the dispute path is symmetric; resolution is not. A buyer who raises a dispute and lets the deadline pass recovers 100% of the amount at no fee. This is not a rounding loss or a fee-maximising edge case: it is the full amount, and the seller’s only recourse is a path that does not exist. Two consequences follow for protocol design. First, an escrow built on this contract is a credit instrument against the seller for the buyer’s dispute decision, so a seller must price that decision out of the consideration, or not trade. Second, the README describes the mechanism more generously (`README.md:286`) than the code behaves, which is the defect a reader relying on the documentation would inherit. The repository documents the behaviour; it does not analyse the consequence. That gap, not the behaviour alone, is the finding.

We state this as a property of the transition relation. We did not execute an attack against it, and no evidence in the audit materials shows anyone exploiting it.

6.3 F3: Truncating fee arithmetic creates a zero-fee region at small amounts

Evidence. The fee is $\lfloor \text{amount} \times \text{fee_bps} / 10,000 \rfloor$ (`lib.rs:199`, `lib.rs:226`) with no minimum-fee term. The suite asserts that releasing 1,000 base units at 80 bps sends 8 to the treasury and 992 to the seller (`tests.rs:193--196`, `tests.rs:849--852`), and that releasing 3,000 base units charges 24 (`tests.rs:794`). The suite also asserts the boundary case: an amount of 1 base

unit at 80 bps produces a fee of 0 and pays the seller the full 1 base unit (`tests.rs:1082--1113`). The cap boundary is enforced on the rate and not on the fee: 1,000 bps is accepted and 1,001 bps raises `InvalidFee` (`tests.rs:921--963`).

Consequence. Equation 1 truncates to zero whenever $\text{amount} \times \text{fee_bps} < 10,000$. At the default rate of 80 bps the largest such amount is $\lfloor (10,000 - 1)/80 \rfloor = 124$ base units; at 1,000 bps it is 9. *This threshold is derived from the formula and the default rate, not itself an executed test result*: the suite exercises only the single-amount case at 1 base unit. The cap is a ceiling on the rate, and the contract has no floor on the fee, so a caller can construct an arbitrarily small escrow that moves value with no protocol fee. Downstream, any arithmetic that assumes a fee was charged for a release — a treasury reconciliation that sums observed fees, or a screening rule keyed on fee presence — has an unmodelled case at amounts at or below the threshold. For the **BREAD** deployment, which the audit materials establish to have 7 decimal places, 124 base units is below the smallest representable trade in any realistic unit; the point is the shape of the mechanism, not this particular bound.

6.4 F4: Mocked authorization masks the authorization coverage the suite appears to report

Evidence. Every test in the suite calls `env.mock_all_auths()`. Under the mock, `caller.require_auth` (`lib.rs:231`) returns success for any caller. The suite contains `test_unauthorized_refund` (`tests.rs:348`), and that test passes — but it passes because of a caller-identity check at `lib.rs:239`, not because an authorization was required and refused. No test in the family asserts that `release_funds` rejects the seller, and no test anywhere in the family exercises a `require_auth` failure.

Consequence. The suite is green, and its green status is not evidence about authorization. The two mechanisms are different: `require_auth` is a signature-authorization gate whose rejection path the mock deletes, while the check at `lib.rs:239` is a caller-identity comparison against a stored party. `test_unauthorized_refund` therefore does provide a real result — only the recorded buyer can `refund_buyer` — but it provides that result through a mechanism it does not name, and it provides no evidence at all about the signature-authorization gate.

This is the finding with the widest reach, because it is about the test infrastructure rather than about this contract. `mock_all_auths` is the correct default for testing business logic, and it is what almost every Soroban test suite uses. A suite can be large, thorough-looking, and entirely green while providing zero evidence about authorization enforcement, and a reviewer reading test counts and names has no signal that distinguishes the two cases. A test whose name asserts a property the harness has disabled is worse than no test, because it is counted as coverage.

7 Threat Model Discussion

This section is analysis, not a report of exploited attacks. The audit performed no penetration test, defined no adversary, and ran no adversarial scenario against the deployed contracts. No evidence in the audit materials indicates that any party exploited any vector below. Each item states what the findings in Section 6 make *reachable or visible* as a property of the code, and stops there.

T1 — Dispute extraction (from F2). The transition relation admits a path from `Funded` to full buyer recovery with no seller payment, taken by raising a dispute and expiring. The cost to the actor is one transaction and the passage of the deadline. The consequence for a protocol built on this contract is a value transfer that the protocol cannot prevent, which is a pricing problem for the seller and a control problem for the integrator.

T2 — Liveness loss in the sibling contract (pactopay). The audit found two liveness defects in the sibling contract: it has no deadline field, so funds can lock indefinitely; and a `Disputed` milestone has no outgoing transition, so funds are unrecoverable in both directions. Neither is covered by a test. These are availability properties, and they are the class of defect the mock-based suite is least likely to surface, because a test that never asserts a deadline or an outgoing transition cannot fail for either.

T3 — Fee-free micro-escrow (from F3). The absence of a minimum fee means the protocol's own fee mechanism has a blind region at small amounts, and the number of distinct escrow records a caller can create and release is not bounded by any fee the protocol collects. The paper makes no claim about whether this is reachable at a volume that matters; no volume measurement exists.

T4 — Assurance inflation (from F4). This is a vector against the *evidence*, not against the contract. A green suite with named tests that do not exercise the property they name is consumed as authorization evidence by reviewers, integrators, and funders who have no way to detect the gap from the test output. The cost of the gap is paid downstream, in an integration that was reviewed against evidence that did not exist.

F1 does not appear in this list as an attack, and should not be read as one. It is a cost-structure property. Its relevance to security is indirect and worth stating precisely: a threat model that budgets effort per operation, guided by a flat per-transaction cost, will spend its attention on the operations the documentation discusses. Finding 6.1 shows that the operation which commits the buyer's money is the expensive one, and Finding 6.2 shows that the operation which decides who is paid is a terminal transition. Both are missed by a per-operation view.

8 Limitations

This section is load-bearing. The findings in Section 6 are defects with consequences; the statements below are absences, and an absence cannot be repaired by argument. A reader deciding whether to rely on this work should read this section before the findings.

8.1 No latency instrumentation exists anywhere

The repository contains no latency instrumentation. A search for `Date.now`, `performance.now`, `elapsed`, `median`, and `p95` returns four hits, all in unrelated UI logic. There is no `performance.now()`, no `elapsed`, no `median`, and no `p95` in the codebase. The README's "< 3.5s median testnet settlement" has no measuring code behind it, and this paper reports no settlement latency, settlement speed, or completion time of any kind. The ledger numbers in Table 3 are ordinal sequence numbers and are not convertible to durations from the available data.

8.2 Sample size

Every measurement in Section 5 is $n = 1$ per step. There are no distributions, no repeated trials, and no confidence intervals. The value 20.6 in Equation 2 is the ratio of two single observations. It is not an estimate of a population ratio, and its reproducibility across runs, amounts, assets, and contract versions is unknown.

8.3 Network

All three escrow proofs are testnet. Nothing in this paper establishes mainnet behaviour, real fee markets, or real liquidity in the assets involved. The BREAD asset in the fee measurement is self-issued by the project, and the USDC proofs reference Circle-issued USDC as represented on a test network, which establishes nothing about the mainnet asset.

8.4 Verification

There is no threat-model adversary, no penetration test, and no formal verification of the contract. Section 7 is analysis. No executed attack is reported anywhere in this paper.

8.5 Untested deployment path

The per-user deployment path — the mechanism the application actually uses — was never executed. `README.md:97` states that it requires a wallet signature. The fee data in Table 3 comes from a different, operator-side path.

8.6 Test-suite scope

The `proofdrop` suite is 100% mocked, runs at $n = 1$, and uses an in-memory store. The `pactopay` TypeScript integration and end-to-end suites fail (Table 4). No repository in the family has a green continuous integration run of record. The two `pactopay` liveness defects are not covered by any test.

8.7 Artifact and version identity

The compiled `escrow.wasm` is not committed, so byte-identity of the deployed artifact cannot be checked from the repository. The fee measurement was taken against `CB7I2GUR...`, while the address the README ships as current is `CAFG7VWEN...`. Findings are scoped accordingly.

8.8 Unsupported claims left in the repository

The repository states that the economics are "85% cheaper" than SWIFT with a flat \$45 charge and a 5.9% spread. These are unsourced hardcoded constants in the source. No comparison against any published price for any payment rail — SWIFT, PayPal, SPEI, PIX — exists in the audit materials. This paper does not repeat those figures as results. The word "cheap" is not used of this system anywhere in this paper, because no comparison was performed.

9 Reproduction

9.1 Commands

```
# Contract suites
cd breadline-protocol && cargo test # 26 passed, 0 failed; sdk 26.1.1
cd proofdrop && npm test # 38 passed, 0 failed
cd pactopay && cargo test # 8 passed, 0 failed; sdk 27.0.6

# Sibling suites, reported as failures
```

Table 5: Artifact provenance. Hashes are recorded where the audit established them; “not committed” means byte-identity is not checkable from the repository.

Artifact	Provenance
lib.rs, tests.rs	source, cited by line
E2E-PROOF.md	fee table, lines 70–95
E2E-USDC-PROOF.md,	escrow proofs
E2E-FEE-PROOF.md	
test_snapshots/	26 committed JSON files
public/escrow.wasm	24,827 bytes; SHA-256
	a648e75a76dae8a8
	b04ab08fc60d2588c
	68a2ffb1e043cdd1
	06ce910a777d2c1;
figures/data/	build artifact <i>not committed</i>
lifecycle_fees.csv	fee table, verbatim
figures/make_figures.py	figure + derivation checks

```
cd pactopay && npx vitest run # failed: 9/11 files, 5/20 tests
cd pactopay && npx playwright test # failed: 8 failed, 4 skipped, 14 passed

# Claimed-but-uninstrumented: returns four hits, all unrelated UI logic
grep -rniE "Date\.now|performance\.now|elapsed|median|p95" .

# Fee table, decoded from the repository's own proof document
sed -n '70,95p' E2E-PROOF.md
```

9.2 Figure and derivation check

The figure is generated from a committed CSV, and the script re-checks every derived quantity in Section 5 against the artifact values before writing output. It exits non-zero if any check fails.

```
cd figures
python3 make_figures.py --verify # print derivations, write nothing
python3 make_figures.py # write lifecycle_fees.pdf
```

9.3 Artifacts

Table 5 lists what the audit read and what could not be checked.

9.4 Build status

This manuscript has been compiled. Three `pdflatex` passes on macOS arm64 with TeX Live from `/Library/TeX/texbin` produce `main.pdf` at nine pages with zero errors, zero undefined references, and zero undefined citations. The manuscript carries no bibliography: it makes no external citation, and Section 9 states that absence as a novelty risk rather than concealing it. The figure is generated by `figures/make_figures.py` from the committed `figures/data/lifecycle_fees.csv` and is typeset as pdf.

10 Conclusion

The design guidance that survives the limitations is narrow and specific.

Price the lifecycle, not the operation. A cost model that assigns one price to a transaction is wrong on this platform by a factor that the contract’s own records make visible. A designer should enumerate the operations a protocol will actually issue, weight them by the resource classes they consume, and treat the most expensive step as the one that scales with the number of escrows. On the evidence here, that step is the one that commits funds.

Make the dispute path symmetric or document it as credit. A dispute mechanism entered by either party and resolved for one is not a dispute mechanism; it is a unilateral recovery right, and the party on the losing end should be told so in the terms a counterparty reads, not only in a design note. If a symmetric resolution path is out of scope, the honest alternative is to remove the dispute entry, not to keep an entry whose only reachable outcome is favourable to one side.

Bound the fee at both ends. A cap without a floor creates a region of amounts on which the protocol collects nothing, and the region is an arithmetic consequence of truncating division rather than a policy decision. Rounding upward, or applying a minimum, removes the region.

Treat the test harness as part of the threat model. A test whose name asserts a property the harness has disabled contributes no evidence, and it contributes that false evidence while counting as coverage. Authorization tests need a harness in which authorization can fail; asserting on the resulting error variant alone is not sufficient, because a different guard can produce the same error.

Read the repository's own records before its prose. Every finding in this paper came from artifacts the project had already committed: its fee table, its proof documents, its snapshot files, its test names. The documentation disagreed with those artifacts on cost and on the dispute path. An audit that reads only the documentation would have found nothing.

None of the four repositories was found to be unsafe, unmaintained, or unworkable, and this paper makes no claim about settlement speed, cost against any payment rail, mainnet behaviour, throughput, or production readiness. The claims are about three specific properties — cost uniformity, dispute symmetry, and test-authorship — and those are the properties the evidence reaches.

References

- [1] **breadline-protocol**. Soroban token-escrow contract repository, Rust, `soroban-sdk` 26.1.1. Audited artifact: contract source `lib.rs`, suite `tests.rs`, 26 committed `test_snapshots` JSON files, committed wasm `public/escrow.wasm` (24,827 bytes, SHA-256 `a648e75a76dae8a8b04ab08fc60d2588c68a2ffb1e043cdd106ce910a777d2c1`; build artifact not committed). Executed suite: `cargo test`, 26 passed, 0 failed, 0.91 s. Retrieval URL not recorded by the audit.
- [2] **breadline-protocol**, `E2E-PROOF.md`. Testnet escrow proof and per-transaction fee record, lines 70–95. Contract `CB7I2GURQDV4Q7YAT2PZAG3ZNFQJ37GI6MWZ4P3W7SUCJZPSHP2G6L6J`; asset `BREAD`, self-issued, 7 decimal places; 20 `BREAD` (200,000,000) base units; 2026-09-25. Source of Table 3 and Figure 1.
- [3] **breadline-protocol**, `E2E-USDC-PROOF.md`. Testnet escrow proof. Contract `CBY6UC4IM5IA6AGRNXOIPENIYNNOL2LJG405QLRLKNWACAEZ2DIUYAWL`; Circle USDC, asset `USDC:GBBD47IF6LWK7P7MDEVSCWR7DPUVW3NY3DTQEVFL4NAT4AQH3ZLLFLA5`; 5 USDC (50,000,000) base units. No date recorded.
- [4] **breadline-protocol**, `E2E-FEE-PROOF.md`. Testnet escrow and fee proof. Contract `CAFG7VWENECKXXSRH64G536IG4CD3ALJ03U6HUAN4P5JPMC4EXOUPHKO`; Circle USDC; 0.05 USDC (500,000) base units. Conservation check $0.0124 + 5.0496 + 0.0004 + 0 = 5.0624$; per-term semantics not established. No date recorded.
- [5] **breadline-protocol**, `README.md`. Project documentation. Cited in this paper only as the source of claims the audit falsifies: “~\$0.00001 per transaction”; “gas is virtually \$0”; “85% cheaper”; “< 3.5 s median testnet settlement”. Line 97 (per-user deployment requires a wallet signature) and line 286 (description of dispute handling) are cited as documentation claims. No timing instrumentation exists behind any figure in this file.
- [6] **proofdrop**. Repository, JavaScript/TypeScript. Executed suite: `npm test`, 38 passed, 0 failed (shared 8/8, api 30/30); 100% mocked, $n = 1$, in-memory store. Retrieval URL not recorded.
- [7] **pactopay**. Repository, Rust with `soroban-sdk` 27.0.6 and TypeScript. Executed: `cargo test`, 8 passed, 0 failed, 0.63 s; `npm vitest run`, failed (9 of 11 files, 5 of 20 tests); `npm playwright test`, failed (8 failed, 4 skipped, 14 passed). Retrieval URL not recorded.
- [8] **niko-sun-stellar**. Repository. Not executed; excluded by a toolchain time ceiling during the audit. No result is reported for this repository and none should be inferred.
- [9] **soroban-sdk 26.1.1**. Rust SDK used to build and test **breadline-protocol**; the version that generates the 26 committed `test_snapshots` JSON files. Recorded from the executed test run. Registry URL and release date not recorded by the audit.
- [10] **soroban-sdk 27.0.6**. Rust SDK used to build and test **pactopay**. Recorded from the executed test run. Registry URL and release date not recorded by the audit.